Ders 5 Genişletilmiş Notu: Hiyerarşik Kümeleme, Uzaklık Fonksiyonları ve Bilgiye Erişim Modelleri

Ders 5 Genişletilmiş Notu: Hiyerarşik Kümeleme, Uzaklık Fonksiyonları ve Bilgiye Erişim Modelleri

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. k-means'in Zayıflıkları ve Hiyerarşik Kümeleme
- 2. Aşağıdan Yukarı (Agglomerative) Kümeleme
- 3. Kümeler Arası Uzaklık Stratejileri
- 4. Sayısal Veri için Uzaklık Fonksiyonları
- 5. İkili ve Nominal Nitelikler için Uzaklık Fonksiyonları
- 6. Metin Dokümanları için Kosinüs Benzerliği
- 7. Kümeleme Değerlendirmesi: Entropi ve Saflık
- 8. Bilgiye Erişim ve Web Araması
- 9. Bilgiye Erişim Modeli ve Mimarisi
- 10. Sorgu Türleri
- 11. Boolean Model ve Vektör Uzayı Modeli
- 12. Diğer Benzerlik Fonksiyonları ve İlgililik Geri Bildirimi

1. k-means'in Zayıflıkları ve Hiyerarşik Kümeleme

Hoca derse önceki haftada kalınan yerden başladı. Kümeleme (clustering) konusunda k-means algoritması ve kümelerin temsil edilmesi görülmüştü. Bu derste önce hiyerarşik kümeleme, uzaklık fonksiyonları (distance functions) ve kümeleme değerlendirmesi (cluster evaluation) işlenecek, ardından bilgiye erişim (information retrieval) bölümüne geçilecekti.

k-means bölümleme tabanlı (partition-based) bir kümeleme algoritmasıdır. Bunun ilk sonucu şudur: Algoritmanın başında küme sayısını (k) algoritmaya vermek gerekir. İkinci zayıflığı, eliptik ya da yuvarlak olmayan şekilleri bulamamasıdır. Hoca ekranda bir şekil çizdi: içte bir küme ve onu halka gibi saran ikinci bir küme. Göz bu iki kümeyi hemen görür. Fakat k-means bu veriye uygulanırsa ideal olmayan, veriyi başka türlü bölen iki küme bulur.

> **Hocanın vurgusu:** k-means'in iki eksiği vardır: eliptik olmayan şekilleri bulamaz ve küme sayısının baştan söylenmesini ister. Hiyerarşik kümeleme bu iki soruna bir çözümdür.

Hiyerarşik kümeleme ayrı bir kümeleme kategorisidir. Dendrogram (dendrogram) adlı bir veri yapısı kullanır. Algoritma veri kümesi üzerinde çalıştıkça bu yapı oluşturulur. Veri kümesindeki kümeler de bu yapı kullanılarak bulunur.

Hiyerarşik kümelemenin iki türü vardır:

- **Aşağıdan yukarı (bottom-up) kümeleme:** Birleştirici (agglomerative) kümeleme olarak da bilinir. Başta veri kümesindeki her veri öğesi bir küme sayılır. Bu tek elemanlı kümeler birleştirilir ve durma koşuluna kadar yukarı doğru devam edilir.
- **Yukarıdan aşağı (top-down) kümeleme:** Bölücü (divisive) kümelemedir. Bütün veri kümesiyle başlar ve tek elemana kadar aşağı iner.

Derste aşağıdan yukarı yöntem işlendi, çünkü yaygın kullanılan hiyerarşik kümeleme odur.

2. Aşağıdan Yukarı (Agglomerative) Kümeleme

2.1 Beş Noktalı Örnek

Hoca iki boyutlu uzayda gösterilmiş beş veri öğesiyle (P_1–P_5) bir örnek verdi. Adımlar şöyledir:

1. Başta her veri öğesi tek elemanlı bir kümedir.
2. Bütün olası kümeler arasındaki uzaklıklar hesaplanır: P_1–P_2, P_1–P_3, P_1–P_4, P_1–P_5, sonra P_2–P_3, P_2–P_4 vb. Kısacası bütün ikili (pairwise) uzaklıklar bulunur.
3. Birbirine en yakın iki küme birleştirilir. Bu örnekte en yakın çift P_1 ile P_2 çıkar. İkisi birleşip iki elemanlı bir küme olur. Hoca buna “1” adını verdi.
4. Artık bir tane iki elemanlı küme (1) ve üç tane tek elemanlı küme (P_3, P_4, P_5) vardır. İkili uzaklıklar yeniden hesaplanır: 1–P_3, 1–P_4, 1–P_5, P_3–P_4 vb.
5. Bir sonraki en yakın çift P_3 ile P_4 çıkar. Bunlar birleşip “2” adlı kümeyi oluşturur.
6. Artık iki tane iki elemanlı küme ve bir tane tek elemanlı küme vardır. Hoca bu adımda 1–P_3, 1–2 ve P_3–2 uzaklıklarının hesaplanacağını, en yakın çiftin 1 ile P_3 çıktığını ve bunların birleşip “3” kümesini oluşturduğunu söyledi.
7. Dendrogram ağacının tamamı kurulmak zorunda olduğu için algoritma her şeyi birleştirir ve sonunda tek bir küme kalır.

Algoritma ilerledikçe bellekte bu veri yapısı (dendrogram) da oluşur. Yani algoritma kümeleme yaparken bir veri yapısı kullanır.

2.2 Dendrogramdan Nihai Kümelemeyi Seçmek

Dendrogram kurulduktan sonra yukarıdan aşağı doğru taranır. Amaç, kümeleme algoritmasının hangi aşamada en iyi sonucu verdiğini bulmaktır.

- Örneğin bir aşamada iki küme vardır. Bu kümelemeye kümeleme değerlendirme yöntemi uygulanır. Kümelemenin başında görülen küme içi uzaklık (intra-cluster distance) ve kümeler arası uzaklık (inter-cluster distance) ölçüleri bir kalite değeri verir.
- Sonra bir alt aşamaya geçilir. Orada üç küme vardır. Aynı kalite ölçüsü uygulanır ve bu aşamanın öncekinden daha iyi olup olmadığına bakılır.
- Böylece dendrogramın farklı aşamaları yukarıdan aşağı incelenir. En iyi değerlendirme sonucunu veren aşama, agglomerative kümelemenin nihai kümelemesi olarak seçilir.

2.3 Algoritmanın Sözde Kodu

Hoca slayttaki sözde kodu (pseudocode) şöyle anlattı: Girdi veri kümesidir. Her veri noktası bir küme yapılır ve bütün ikili uzaklıklar hesaplanır. Sonra döngüye girilir. Birbirine en yakın iki küme bulunur ve birleştirilip yeni bir küme oluşturulur. Yeni kümeden diğer bütün kümelere olan uzaklık hesaplanır. Döngünün başına dönülür ve tek küme kalana kadar tekrarlanır. Döngü bittiğinde dendrogram kurulmuş olur.

```
Agglomerative(D)
  D'deki her veri noktasını bir küme yap
  bütün ikili uzaklıkları hesapla
  tekrarla
    birbirine en yakın iki kümeyi bul
    bunları birleştirip yeni küme c oluştur
    c'den diğer bütün kümelere uzaklığı hesapla
  tek küme kalana kadar
```

Sözde kodda tek elemanlı kümeler arasındaki uzaklık bulunuyor. Fakat kümeler iki, üç ya da daha çok elemanlı da olabilir. Bu yüzden hem bir uzaklık fonksiyonuna hem de çok elemanlı kümeler arasında uzaklığı hesaplama yöntemine ihtiyaç vardır.

3. Kümeler Arası Uzaklık Stratejileri

Çok elemanlı kümeler arasındaki uzaklık için dört yöntem anlatıldı: tek bağlantı (single link), tam bağlantı (complete link), ortalama bağlantı (average link) ve merkez (centroid).

3.1 Tek Bağlantı (Single Link)

İki küme arasındaki uzaklık, iki kümenin birbirine en yakın iki öğesi arasındaki uzaklıktır.

- Örnekte iki elemanlı küme 1 (P_1, P_2) ile tek elemanlı P_3 arasındaki uzaklık isteniyor. Küme 1'in P_3'e en yakın öğesi P_2'dir. P_3 zaten tek elemanlıdır. Uzaklık P_2–P_3 uzaklığıdır.
- İkinci örnekte üç elemanlı küme 3 (P_1, P_2, P_3) ve iki elemanlı küme 2 (P_4, P_5) var. İki ayrı kümeden birbirine en yakın iki öğe P_2 ve P_5'tir. Kümeler arası uzaklık P_2–P_5 uzaklığıdır.

3.2 Tam Bağlantı (Complete Link)

İki ayrı kümeden birbirine en uzak iki nokta seçilir. Aynı örnekte bunlar P_1 ve P_4'tür. İki küme arasındaki uzaklık P_1–P_4 uzaklığıdır.

3.3 Ortalama Bağlantı (Average Link)

İki küme arasındaki bütün ikili uzaklıklar bulunur ve ortalaması alınır. Üç elemanlı küme 3 ile iki elemanlı küme 2 için P_5–P_1, P_5–P_2, P_5–P_3, P_4–P_1, P_4–P_2, P_4–P_3 olmak üzere $3 \times 2 = 6$ uzaklık hesaplanır (çarpım ara adım eklendi). Bu altı uzaklığın ortalaması kümeler arası uzaklıktır.

3.4 Merkez (Centroid)

Her kümenin merkez noktası (centroid) hesaplanır. İki küme arasındaki uzaklık, iki merkez arasındaki uzaklıktır.

3.5 Hangi Strateji Seçilmeli?

Kümeleme algoritmasını çalıştırırken en iyi sonucu verecek uzaklık yöntemini seçmek gerekir. Fakat başta hangisinin en iyi olduğu bilinmez. Bu yüzden elimizdeki bütün yöntemler veri kümesi üzerinde denenir. Algoritma birinci yöntemle çalıştırılıp bir kümeleme sonucu bulunur, ikinci yöntemle çalıştırılıp başka bir sonuç bulunur. Sonra her denemenin kümeleme kalitesi karşılaştırılır ve hangi yöntemin kullanılacağına böyle karar verilir.

Bu dört yöntem yalnızca uzaklık bulunurken hangi veri noktalarının kullanılacağını söyler. Uzaklığın hangi uzaklık fonksiyonuyla ölçüleceğini söylemez. Uzaklık fonksiyonu ayrı bir seçimdir.

4. Sayısal Veri için Uzaklık Fonksiyonları

Uzaklık fonksiyonları veri türüne göre ayrılır. Sayısal (numeric) veride niteliklerin değerleri gerçek sayıdır. Nominal veride değerler kategorik ya da ayrıktır (discrete).

4.1 Minkowski Uzaklığı ve Özel Durumları

Slayttaki Minkowski uzaklığı birçok uzaklık ölçüsünün genelleştirilmiş, soyut hâlidir. r boyutlu (nitelikli) iki veri noktası x_i ve x_j için:

$$dist(x_i, x_j) = \left(\sum_{f=1}^{r} |x_{if} - x_{jf}|^h \right)^{1/h}$$

Bu ölçü farklı biçimlerde özelleştirilir:

- $h = 2$ konursa Öklid uzaklığı (Euclidean distance) elde edilir. Kümeleme algoritmalarında en sık kullanılan uzaklık ölçülerinden biridir.
- $h = 1$ konursa aynı formül Manhattan uzaklığı (Manhattan distance) adını alır.

4.2 Ağırlıklı Öklid, Kareli Öklid ve Chebyshev

- **Ağırlıklı Öklid (weighted Euclidean):** Öklid uzaklığındaki farklara ağırlık verilmek istenirse kullanılır:

$$dist(x_i, x_j) = \sqrt{w_1(x_{i1} - x_{j1})^2 + w_2(x_{i2} - x_{j2})^2 + \cdots + w_r(x_{ir} - x_{jr})^2}$$

- **Kareli Öklid (squared Euclidean):** Hesapta birbirinden uzak olan veri noktalarına daha çok önem verilmek isteniyorsa kullanılır. Hocanın anlattığı gibi, her nitelikteki farkın karesi alınır:

$$dist(x_i, x_j) = (x_{i1} - x_{j1})^2 + (x_{i2} - x_{j2})^2 + \cdots + (x_{ir} - x_{jr})^2$$

- **Chebyshev uzaklığı:** İki veri noktası herhangi bir nitelikte farklıysa onları farklı saymak isteniyorsa kullanılır. Bir veri öğesinin birçok boyutu (niteliği) olabilir. Her nitelik için fark hesaplanır ve bu farkların en büyüğü iki nokta arasındaki uzaklık olarak atanır:

$$dist(x_i, x_j) = \max\left(|x_{i1} - x_{j1}|, |x_{i2} - x_{j2}|, \ldots, |x_{ir} - x_{jr}|\right)$$

4.3 Strateji ile Fonksiyonun Parametrik Seçilmesi

Hoca sözde koda geri döndü. "Bütün ikili uzaklıkları hesapla" adımı uygulanırken iki seçim yapılır:

- Bir uzaklık hesaplama stratejisi: tek bağlantı, tam bağlantı, ortalama bağlantı ya da merkez.
- Bu stratejinin içinde kullanılacak uzaklık fonksiyonu. Örneğin tek bağlantı ile Chebyshev uzaklığı ya da tek bağlantı ile Öklid uzaklığı birlikte kullanılabilir.

Hangi kombinasyonun en iyi sonucu vereceği bilinmez. Bu yüzden algoritma bu iki seçime göre parametrik yazılmalı, farklı stratejiler farklı uzaklık fonksiyonlarıyla birlikte denenmelidir.

> **Hocanın vurgusu:** En iyi stratejiyi ve en iyi uzaklık fonksiyonunu baştan bilemeyiz. Hepsi parametre olmalı; farklı kombinasyonlar denenip kümeleme kalitesi karşılaştırılmalıdır.

Buraya kadar görülen Chebyshev, kareli Öklid, ağırlıklı Öklid, Manhattan ve Öklid uzaklıklarının hepsi, veri noktasının nitelik değerlerinin gerçek sayı olduğu sayısal verilere uygulanır.

5. İkili ve Nominal Nitelikler için Uzaklık Fonksiyonları

Bazı durumlarda veri noktalarının nitelikleri ikili (binary) ya da kategorik değerli olabilir. Hocanın örneği cinsiyet niteliğiydi: iki durumu olan ikili bir nitelik. Böyle iki veri noktası arasındaki uzaklığı ölçmek için de uzaklık ölçüleri vardır.

5.1 Karışıklık Matrisi

Bu ölçüleri uygulamak için karışıklık matrisi (confusion matrix) adlı bir matris kullanılır. Karışıklık matrisi eğitmenli öğrenme (supervised learning) algoritmalarının kalitesini ölçerken de kullanılır, ama burada uzaklık değerini hesaplamak için kullanılıyor. İki veri noktası x_i ve x_j için nitelikler dört gruba sayılır:

	$x_j = 1$	$x_j = 0$
$x_i = 1$	a	b
$x_i = 0$	c	d

	$x_j = 1$	$x_j = 0$

- a: iki noktada da 1 olan nitelik sayısı.
- b: birinci noktada 1, ikincide 0 olan nitelik sayısı.
- c: birinci noktada 0, ikincide 1 olan nitelik sayısı.
- d: iki noktada da 0 olan nitelik sayısı.

b ve c uyuşmazlıkları (mismatch) gösterir. Dolayısıyla $b + c$ toplam uyuşmazlık sayısıdır, $a + b + c + d$ ise toplam nitelik sayısıdır.

5.2 Basit Eşleşme Katsayısı (Simetrik İkili Nitelikler)

Basit eşleşme katsayısı (simple matching coefficient), uyuşmayan değerlerin oranıdır. Payda uyuşmayan nitelik sayısı, paydada toplam nitelik sayısı vardır:

$$dist(x_i, x_j) = \frac{b + c}{a + b + c + d}$$

Slayttaki örnekte iki veri noktası ve yedi ikili nitelik vardı:

	1	2	3	4	5	6	7
x_1	1	1	1	0	1	0	0
x_2	0	1	1	0	0	1	0

Çözüm:

- Uyuşmazlıklar: 1. nitelik (1 ve 0), 5. nitelik (1 ve 0), 6. nitelik (0 ve 1). Hoca bunları tek tek sayarak üçe ulaştı. Böylece $b = 2$, $c = 1$, $b + c = 3$.
- İkisinde de 1 olanlar: 2. ve 3. nitelik, yani $a = 2$. İkisinde de 0 olanlar: 4. ve 7. nitelik, yani $d = 2$ (ara adım eklendi).
- Payda toplam nitelik sayısıdır: $2 + 2 + 1 + 2 = 7$.
- Sonuç: $dist(x_1, x_2) = \frac{3}{7} \approx 0{,}429$.

  Ders sonrası not: Hoca 00:40:21 civarında "paydada uyuşmazlık sayısını yazmalısınız" dedi. Doğrusu uyuşmazlık sayısı paya (numerator) yazılır; paydada toplam nitelik sayısı vardır. Hemen ardından hesabı doğru biçimde yaptı.

5.3 Jaccard Katsayısı (Asimetrik İkili Nitelikler)

Bazen nitelik ikili olsa da bir durumun diğerinden daha önemli olduğu söylenir. Bu tür niteliklere asimetrik ikili nitelik (asymmetric binary attribute) denir. Bu durumda Jaccard katsayısı (Jaccard coefficient) kullanılır.

Jaccard katsayısı basit eşleşme katsayısıyla aynıdır. Tek fark paydadadır: Önemsiz durum sayılmaz. Örnekte 1 durumu 0'dan daha önemlidir. Bu yüzden iki veri öğesinde de değerin 0 olduğu nitelikler, yani d, paydaya katılmaz:

$$dist(x_i, x_j) = \frac{b + c}{a + b + c}$$

Aynı örnekte pay yine uyuşmazlık sayısı olan 3'tür. Paydada iki noktada da 0 olan nitelikler dışındaki bütün nitelikler vardır: $a + b + c = 2 + 2 + 1 = 5$. Sonuç $\frac{3}{5} = 0{,}6$'dır.

Ders sonrası not: Hoca paydayı 6 aldı ve sonucu $\frac{3}{6}$ dedi. Örnekte iki noktada da 0 olan nitelik sayısı $d = 2$'dir (4. ve 7. nitelik). Bu yüzden payda $7 - 2 = 5$, sonuç $\frac{3}{5} = 0{,}6$ olur (python ile doğrulandı).

Ek bilgi (derste anlatılmadı): Derste "Jaccard katsayısı" adıyla verilen $\frac{b+c}{a+b+c}$ aslında bir uzaklıktır. Jaccard benzerliği $\frac{a}{a+b+c}$'dir ve bu uzaklık $1 - \frac{a}{a+b+c}$'ye eşittir. Örnekte benzerlik $\frac{2}{5} = 0{,}4$, uzaklık $0{,}6$ olur. Kaynaklarda iki adlandırma da görülebilir.

5.4 Nominal Nitelikler

Nitelik değerleri ikiden fazla durumlu olabilir. Hocanın örnekleri haftanın günü (pazartesi, salı, … pazar) ve renkti (yeşil, kırmızı, sarı). Bunlar nominal, yani kategorik değerli niteliklerdir. Bu durumda yine basit eşleşme mantığı kullanılır: payda uyuşmazlık sayısı, paydada toplam nitelik sayısı vardır.

$$dist(x_i, x_j) = \frac{r - q}{r}$$

Burada r toplam nitelik sayısı, q eşleşen nitelik sayısıdır. $r-q$ uyuşmazlık sayısını verir. Formül ikili nitelikler için verilen basit eşleşme katsayısıyla aynı fikirdir; bu kez ikiden fazla durumlu nitelikler için uygulanır.

6. Metin Dokümanları için Kosinüs Benzerliği

Metin dokümanları için de uzaklık fonksiyonları vardır. Hoca bu konunun sonraki derslerde çok konuşulacağını söyledi. Örneğin dokümanlar ile sorgular (query) arasındaki uzaklık karşılaştırılacak. Bunun için önce dokümanların vektöre çevrilmesi gerekir. Dokümanın vektöre nasıl çevrileceği ileride anlatılacaktı. Şimdilik kelime çantası (bag of words) adlı bir yöntemle bunun yapılabildiği varsayıldı.

Elimizde birinci dokümana ve ikinci dokümana ait iki vektör olsun. Her iki dokümanı da vektörle temsil ettiğimiz için aralarındaki uzaklığı hesaplamamız gerekir. Bu durumda kosinüs benzerliği (cosine similarity) kullanılabilir. Kosinüs benzerliği iki vektör arasındaki uzaklığı hesaplamak için kullanılan bir ölçüdür. Ayrıntısı bir sonraki bölümde (Bölüm 11) verildi.

Hoca bu kısmı şöyle toparladı: Gerçek sayılar uzayındaki uzaklık fonksiyonları, ikili nitelikler için iki uzaklık ölçüsü, nominal nitelikler için bir uzaklık ölçüsü ve dokümanlar için kosinüs benzerliği görüldü. Kümeleme yaparken bu uzaklık ölçüleriyle uzaklık hesaplama stratejilerinin farklı kombinasyonları denenmelidir.

7. Kümeleme Değerlendirmesi: Entropi ve Saflık

Hangi kümeleme sonucunun seçileceğine karar vermek için bir kümeleme değerlendirme yöntemi gerekir.

7.1 Etiket Yoksa: Küme İçi ve Kümeler Arası Uzaklık

Bu ölçüler kümelerin sıkılığını (compactness) ve ayrıklığını (isolation) verir.

- **Küme içi uzaklık (intra-cluster distance):** Bir kümedeki bütün olası veri öğeleri arasındaki uzaklık hesaplanır. Bu bir sayı verir. İşlem kümeleme sonucundaki bütün kümeler için yapılır. Sonuç kümenin sıkılığını gösterir.
- **Kümeler arası uzaklık (inter-cluster distance):** Bu kez kümeleme sonucundaki bütün olası kümeler arasındaki uzaklık hesaplanır. Sonuç kümelerin birbirinden ne kadar ayrık olduğunu gösterir.

  Ders sonrası not: Hoca 00:49:35–00:50:56 arasında iki ölçüyü de "inter-cluster" diye söyledi. Sıkılığı gösteren, küme içindeki öğeler arasında hesaplanan ölçü küme içi (intra-cluster) uzaklıktır. Dersin başında (00:20:30) ve sonunda (00:58:23) aynı ayrım intra/inter olarak geçiyor.

Veri öğeleri hakkında başka bir şey bilinmiyorsa bu iki ölçü kullanılır. Kümeleme problemlerinde genelde etiketli veri yoktur; hangi öğenin hangi kümeye ait olduğu baştan bilinmez. Bu yüzden çoğunlukla bu iki uzaklık ölçüsüne başvurulur.

7.2 Etiket Varsa: Entropi, Saflık, Precision, Recall, F-score

Bazen etiketli veri (ground truth) bulunabilir, yani hangi öğenin hangi kümeye ait olduğu bilinir. O zaman bu etiketler kullanılarak entropi (entropy), saflık (purity), precision, recall ve F-score gibi ölçülerle değerlendirme yapılabilir. Derste entropi ve saflık işlendi.

> **Hocanın vurgusu:** Entropi ve saflık formüllerini ezberlemenizi istemiyorum; değerlerin nasıl yorumlanacağını bilmeniz yeterli.
>
> **Ek bilgi (derste anlatılmadı):** Bing Liu'nun kitabındaki tanımlarla, D_i kümesinde c_j sınıfının oranı $\Pr_i(c_j)$ ise kümenin entropisi $entropy(D_i) = -\sum_j \Pr_i(c_j) \log_2 \Pr_i(c_j)$, saflığı $purity(D_i) = \max_j \Pr_i(c_j)$'dir. Toplam değer, her kümenin değerinin küme boyutuyla ağırlıklandırılmış ortalamasıdır: $\sum_i \frac{|D_i|}{|D|} entropy(D_i)$. Aşağıdaki tablo değerleri bu tanımlarla python'da yeniden hesaplandı.

7.3 900 Dokümanlı Örnek

Veri kümesinde 900 veri öğesi vardır ve hepsinin etiketi bilinmektedir: 300'ü bilim (science), 300'ü spor (sports), 300'ü siyaset (politics). Kümeleme algoritması üç küme bulmuştur.

İdeal sonuç şu olurdu: Birinci küme aynı etiketli 300 öğeyi, ikinci küme diğer 300'ü, üçüncü küme kalan 300'ü içerir. Sonuç böyle çıkmadı:

Küme	Science	Sports	Politics	Toplam	Entropi	Saflık
1	250	20	10	280	0,589	0,893
2	20	180	80	280	1,198	0,643
3	30	100	210	340	1,257	0,617
Toplam	300	300	300	900	1,031	0,711

Yorum:

- **Entropi belirsizliği (uncertainty) gösterir.** Belirsizlik yüksekse entropi büyüktür, düşükse küçüktür. Birinci kümede öğelerin çoğu tek bir etikete (science) aittir. Bu kümeden rastgele bir öğe seçilse büyük olasılıkla science etiketli çıkar. Belirsizlik düşüktür, dolayısıyla entropi düşüktür.
- **Saflık homojenliği gösterir.** Kümedeki öğelerin çoğu aynı etikete sahipse saflık yüksektir; dağılım ne kadar homojense saflık o kadar büyüktür. Birinci kümede saflık yüksek, entropi düşük olmalıdır.
- **Üçüncü küme:** Buradan rastgele seçilen bir öğe siyaset etiketli de olabilir, spor etiketli de. Belirsizlik birinci kümeden büyüktür, bu yüzden entropi daha yüksektir. Dağılım da birinci küme kadar homojen değildir. Hoca kabaca öğelerin üçte birinin spor, üçte ikisinin siyaset etiketli olduğunu söyledi ($100/340 \approx 0{,}29$, $210/340 \approx 0{,}62$). Bu yüzden saflık düşüktür.

Hesap kontrolü (ara adımlar eklendi): Birinci küme için oranlar $250/280$, $20/280$, $10/280$'dir. Saflık $250/280 \approx 0{,}893$'tür. Entropi $-\sum p \log_2 p \approx 0{,}5896$'dır. Üçüncü kümenin entropisi $\approx 1{,}2577$, saflığı $210/340 \approx 0{,}6176$'dır. Tablodaki 0,589, 1,257 ve 0,617 değerleri bu sayıların kesilmiş hâlidir.

> **Ders sonrası not:** Hoca birinci kümede "üç tanesi sports" dedi. Tablodaki değer 20'dir. Kümenin toplamı 280 ve entropisi 0,589 ancak 20 ile tutar; 3 alınsaydı entropi büyük olasılıkla yaklaşık 0,32 çıkardı (python ile hesaplandı).

Hoca bu kısmı şöyle kapattı: Etiketli veri varsa entropi ve saflık, ayrıca precision, recall ve F-score kullanılabilir. Etiketli veri yoksa küme içi uzaklık (kümelemenin ne kadar sıkı olduğu) ve kümeler arası uzaklık (kümelerin ne kadar ayrık olduğu) hesaplanmalıdır.

8. Bilgiye Erişim ve Web Araması

Aradan sonra Chapter 6'ya, bilgiye erişim ve web aramasına (information retrieval and web search) geçildi. Hoca slaytların PDF olarak ders sayfasında bulunduğunu, kitabın web sitesinde

de bölümlerin ve ders slaytlarının yer aldığını hatırlattı.

8.1 Web Madenciliğindeki Yeri

Web veri madenciliğinin (web data mining) üç ana kategorisi vardır:

- **Web içerik madenciliği (web content mining):** Dokümanları bulmak, indekslemek, indekslenmiş dokümanlar üzerinde arama yapmak ve sonuçları kullanıcıya döndürmekle ilgilenir. Dokümanların içeriğinin indeks oluşturmak için nasıl işleneceği ve kullanıcı sorgularıyla indeks tablosunda nasıl arama yapılacağı bu kısma girer.
- **Yapı madenciliği (structure mining):** Bir sonraki bölümde, dokümanlar arasındaki bağlantı yapısının arama sonuçlarını iyileştirmek için nasıl kullanılacağı görülecek.
- **Web kullanım madenciliği (web usage mining):** Dersin son kısmında, kullanıcı ile web sitesi etkileşimlerinden oluşan log dosyalarının kullanıcı davranışını anlamak için nasıl analiz edileceği görülecek.

Bilgiye erişim bölümü web içerik madenciliğinin ilk bölümüdür. Web içerik madenciliği büyük ölçüde metin madenciliğidir (text mining): Metin dokümanları üzerinde veri madenciliği yapılır, metin üzerinde arama yapılabilecek şekilde işlenir. Bunun için bilgiye erişim yöntemleri kullanılır. Bu yüzden önce bilgiye erişimin ne olduğunu ve veritabanından erişimden nasıl ayrıldığını anlamak gerekir.

8.2 Bilgiye Erişim Nedir?

Bilgiye erişim (information retrieval, IR), dokümanlar arasından bilgi bulma çalışmasıdır. IR'de veri yapısızdır (unstructured): metin dokümanları, PDF dokümanları, Word dokümanları vb. Sorgu ise anahtar kelime (keyword) ya da anahtar kelime grupları ile ifade edilir. Bu sorgular dokümanlara uygulanır ve sorguyla eşleşen dokümanlar bulunmaya çalışılır. IR'nin işi, kullanıcının aradığını bulabilmesi için dokümanları sorgularla eşleştirmektir.

IR şu aşamaların hepsiyle ilgilenir:

- **Verinin toplanması (acquisition):** Verinin nasıl toplanıp merkezi bir veritabanına getirileceği.
- **Düzenlenmesi (organization):** Veri üzerinde arama yapılabilmesi için düzenlenmesi, yani indeks tablosunun oluşturulması.
- **Saklanması (storage):** Büyük ölçekli indeks tablolarının çok sayıda düğümde (node) nasıl saklanacağı.
- **Getirilmesi (retrieval):** Anahtar kelimelerin indeks tablosuna nasıl uygulanacağı.
- **Dağıtılması (distribution):** Getirilen sonuçların kullanıcıya nasıl dağıtılacağı.

Kısacası IR, bilgisayar biliminin doküman koleksiyonlarından bilgi getirmeyi inceleyen bir parçasıdır. Verilen bir sorguya göre getirilen dokümanlar kullanıcının bilgi ihtiyacını (information need) karşılamalıdır.

Hoca ekranda farklı IR motorları gösterdi. IEEE Xplore ve ACM'nin arama motorları, kullanıcının doküman kümeleri üzerinde karmaşık sorgular kurabilmesi için birçok yetenek sunar. Google basit bir kullanıcı arayüzü sağlar; gelişmiş bir sürümü de vardır ama genelde basit arayüz kullanılır.

8.3 Veri, Bilgi, Bilgi Birikimi, Bilgelik

Hoca daha önce gösterdiği bir slayda döndü. IR terminolojisinde:

- İşlenen ham web sayfaları veridir (data).
- Sorgu uygulandıktan sonra arama motorunun gösterdiği ilk 10 sonuç listesi bilgidir (information).
- Sonuç listesindeki her bağlantı açılıp içerik analiz edildikçe sorgu hakkında zihinde bir resim oluşur. Bu bilgi birikimidir (knowledge).
- Bilgiden çıkarılan aynı bilgi birikimi birçok kişide ortaklaşırsa bilgelik (wisdom) olur.

8.4 Bilgiye Erişim ile Veritabanı Erişimi Farkı

IR'de doküman kümesine bir sorgu uygulanır ve sonuçlar gelir. Doküman kümesinde de sorguda da yapı yoktur; sorgu yalnızca bir anahtar kelime kümesidir. Sonuç kümesi de HTML, PDF ve Word dokümanları içerebilir, yani o da yapısızdır.

Veritabanından veri erişiminde ise veri yapılıdır, çünkü tablolarda tutulur. Her tablonun nitelikleri, her niteliğin değerleri ve veri tipleri vardır. Sorgular da yapılıdır; sorguyu kurmak için SQL yapısına uymak gerekir. Veri, sorgu ve sonuç kümesinin hepsi yapılıdır. Bu yüzden IR, veritabanından erişimden büyük ölçüde farklıdır.

> **Hocanın vurgusu:** Bilgiye erişimde hem dokümanlar hem sorgu yapısızdır; veritabanında veri, SQL sorgusu ve sonuç yapılıdır.

8.5 Web Aramasının Kendine Özgü Sorunları

IR'nin en iyi bilinen örneği web aramasıdır (web search). Web araması da doküman kümeleri üzerinde arama yapar, fakat IR'den farklı kendine özgü özellikleri vardır:

- **Hiperlinkler:** Web aramasında HTML dokümanları arasındaki hiperlinklerden yararlanılabilir. Klasik IR'de dokümanlar birbirini göstermez. Web'de ise dokümanlar hiperlinklerle birbirine işaret eder ve bu bağlantılar kullanılabilir.
- **Ölçek:** Çok sayıda dokümanla uğraşıldığı için dokümanları toplamak, saklamak ve güncellemek çok daha zordur.
- **Yanıt süresi:** Web'de aynı anda birçok kişi arama yapar. Kullanıcıya hızlı yanıt vermek ve çok sayıda kullanıcıya ölçeklenmek gerekir.
- **Spam:** Bu, e-posta hesabında görülen spamden farklıdır. Bir web sitesine sayfanın konusuyla ilgisiz anahtar kelimeler eklenirse sayfa spam sayfasına dönüşür. Amaç, sitenin genel konusunu temsil etmeyen kelimelerle farklı aramalarda üst sıralarda görünmektir. Arama motorlarının bu ilgisiz kelimeleri fark edip spam sorunuyla başa çıkabilmesi gerekir.

Bu özellikler yüzünden web araması bir IR motoru sayılsa da çözülmesi gereken özel sorunları vardır ve bu onu klasik IR motorlarından ayırır.

9. Bilgiye Erişim Modeli ve Mimarisi

9.1 IR Modelinin Tanımladığı Dört Şey

Her IR motorunun bir IR modeli vardır. Model, verilen bir sorgunun doküman kümesine uygulanmasını sağlar ve dört şey tanımlar:

- **Sorgu dili (query language):** Basit anahtar kelimeler, bir ifade (phrase) ya da doğal dil cümlesi olabilir. Her IR motoru sorgu oluşturmak için farklı kullanıcı arayüzleri sunar.
- **Sorgunun iç temsili (internal representation):** Kullanıcı arayüzüyle oluşturulan sorgu içeride bir biçimde temsil edilir.
- **Dokümanın iç temsili:** Sorgunun uygulanacağı doküman kümesinin de bir iç temsili olmalıdır.
- **Karşılaştırma fonksiyonu:** Sorgu ile dokümanı karşılaştırıp sorgunun doküman kümesindeki dokümanlarla eşleşip eşleşmediğini bulur. Bu fonksiyon aynı zamanda sıralama fonksiyonudur (ranking function).

İşleyiş şöyledir: Kullanıcı, sorgu dilinin tanımladığı arayüzle sorguyu girer. Model sorguyu kendi iç temsiline çevirir. Doküman koleksiyonundaki dokümanlar da bir temsil yöntemiyle temsil edilmiştir. Karşılaştırma fonksiyonu sorguyu dokümanlarla karşılaştırır, eşleşen dokümanları bulur ve sonuçları kullanıcıya gösterir.

9.2 IR Mimarisi

Hoca IR mimarisinin modüler diyagramını şöyle anlattı:

1. Her IR motorunda bir indeksleyici (indexer) vardır. İndeksleyici doküman koleksiyonunu işler ve bir indeks oluşturur. Arama bu indeks üzerinde yapılır.

2. Kullanıcı, IR motorunun sunduğu arayüzleri kullanarak sorgusunu kurar. Bundan çalıştırılabilir bir sorgu (executable query) oluşur.
3. Erişim sistemi (retrieval system) çalıştırılabilir sorguyu alır ve iç temsile çevirir.
4. Sorgunun iç temsili, doküman indeksindeki dokümanların iç temsiliyle karşılaştırılır.
5. Erişim sisteminden gelen sonuçlar kullanıcıya gösterilir. Genelde ilk 10 sonuç gösterilir.

10. Sorgu Türleri

Sorgu dili genelde IR modelinin bir parçasıdır. IR'de farklı türde sorgular yapılabilir:

- **Anahtar kelime sorgusu (keyword query):** Arama motoruna bir anahtar kelime ya da kelime grubu girilir ve aramayı bizim yerimize tamamlaması istenir. Google'ı çoğunlukla bu şekilde kullanırız.
- **Boolean sorgu (Boolean query):** AND, OR ve NOT işleçleriyle (operator) kurulur. Hoca IEEE Xplore'un gelişmiş arama ekranını gösterdi: Bir arama terimi girilip hangi doküman alanında aranacağı seçilir, Boolean işleçle başka bir terim ve alan eklenir. Arama oluşturulunca ekran son sorgunun çalıştırılabilir hâlini gösterir ve bu sorgu doküman indeksine uygulanır. Boolean sorguda sonuçlar yalnızca sorguyla tam eşleşen dokümanlardır. Örneğin "web AND mining AND NOT text" sorgusu web ve mining kelimelerini içeren ama text içermeyen dokümanları getirir. Sorguyu doğru (true) yapan dokümanlar sonuçta görünür, yanlış (false) yapanlar görünmez.
- **İfade sorgusu (phrase query):** Kelime grubu tırnak içine alınırsa ifade sorgusu olur. Örneğin "web data mining" yazılırsa bu üç kelimenin tam bu sırayla, art arda geçtiği ifade aranır.
- **Yakınlık sorgusu (proximity query):** Bazı arama motorları sunar. Tırnak gerekmez. web, data ve mining kelimelerinin yan yana olması gerekmez, ama dokümanda birbirine yakın olmaları beklenir. Aynı cümlede ya da aynı paragrafta olabilirler. Kelimeler birbirine ne kadar yakınsa doküman sıralamada o kadar üst sıraya çıkar.
- **Tam doküman sorgusu (full document query):** Google görsel aramasında olduğu gibi arama motoruna bir görsel verilir ve benzer görseller bulunur. Ya da bütün bir doküman verilir ve onunla ilgili dokümanlar bulunur. Bazı intihal (plagiarism) denetim siteleri dokümanın tamamını ister, kendi çevrimiçi veritabanlarında arar ve eşleşen dokümanları bulur.
- **Doğal dil sorusu (natural language question):** Hocanın Google için örneği "who is Joe Biden" sorusuydu. Arama motoru yalnızca sonuçları listelemez, sorunun cevabını da verir. Yani güncel arama motorları doğal dil sorularını da işler. Doğrudan doğal dil sorusu alan arama motorları da vardır.

IR motorunun kullanıcı arayüzü bu farklı sorgu türlerini girebilmek için farklı arayüzler sunar. Motor sorgu dilinden çalıştırılabilir sorguyu oluşturur, ardından bu sorgu iç temsile çevrilir.

11. Boolean Model ve Vektör Uzayı Modeli

IR motorları IR modellerini kullanır. Farklı modeller vardır: Boolean model, vektör uzayı modeli (vector space model) ve istatistiksel dil modeli (statistical language model). Bu modellerin hepsi doküman ve sorgunun nasıl temsil edileceğini ve bir dokümanın kullanıcı sorgusuyla ilgililiğinin (relevance) nasıl tanımlanacağını belirler; yani sorgu ile dokümanı karşılaştırıp benzerliği bulan karşılaştırma fonksiyonunu da tanımlar. Bu derste Boolean model ve vektör uzayı modeli işlendi.

11.1 Kelime Çantası ve Sözlük

Boolean modelde her doküman bir kelime çantası (bag of words) olarak temsil edilir. Bu adın sebebi, her dokümandan onu temsil edecek anahtar kelimelerin çıkarılmasıdır.

Bir doküman kümesi D olsun. Her dokümanı kelime çantası olarak temsil etmek için:

1. Önce bütün dokümanlar için bir sözlük (vocabulary) oluşturulur. Birinci doküman ayrıştırılır (parse) ve ayırt edici kelimeleri bulunur. Aynı işlem ikinci, üçüncü, dördüncü dokümanda, son dokümana kadar yapılır. Çıkarılan terimlerin hepsi sözlüğü oluşturur.

2. Bu sözlükle kümedeki bir doküman, örneğin d_j, bir vektör olarak temsil edilir. Vektörün boyutu sözlüğün boyutuna ($|V|$) eşittir.
3. Boolean modelde d_j için anahtar kelime dokümanda geçiyorsa ilgili konuma 1, geçmiyorsa 0 yazılır. Bu vektörler kümedeki her doküman için oluşturulur.

11.2 Boolean Model: Tam Eşleşme

Boolean modelde de karşılaştırma fonksiyonu gerekir ve hem sorgu hem dokümanlar temsil edilmelidir. Kullanıcının girdiği anahtar kelimeler Boolean işleçlerle birleştirilir. IEEE ve ACM bu işleçleri sunar, böylece Boolean modele dayalı sorgu kurulabilir. Böyle bir sorgu kurulduğunda, ifadeyi doğru yapan dokümanlar aranır. İfadeyi doğru yapan bütün dokümanlar sonuçlarda yer alır, diğerleri almaz. Bu tür erişime tam eşleşme (exact match) denir.

11.3 Vektör Uzayı Modeli ve TF-IDF

Vektör uzayı modelinde yine kelime çantası temsili kullanılır. Fakat terimler için 1 ve 0 yerine farklı bir değer yazılır. Bu değer TF-IDF ölçüsüne göre hesaplanır. TF-IDF iki parçadan oluşur: terim frekansı (term frequency, TF) ve ters doküman frekansı (inverse document frequency, IDF).

Terim frekansı (TF): Bir kelimenin bir doküman içindeki frekansıdır. i kelimesinin j dokümanında kaç kez geçtiği sayılır (f_{ij}). Bu değer, o dokümanda en çok geçen kelimenin frekansına bölünür:

$$tf_{ij} = \frac{f_{ij}}{\max\{f_{1j}, f_{2j}, \dots, f_{|V|j}\}}$$

Hoca bunu Türkçe şöyle tekrarladı: Pay, i kelimesinin j dokümanındaki tekrar sayısıdır. Paydada, j içindeki diğer kelimelerin frekansları bulunur ve bunların en büyüğü alınır. Bu oran, i kelimesinin j dokümanındaki önemini, en çok tekrar eden kelimeyle karşılaştırılmış tekrar yoğunluğunu verir. Değer yüksekse i kelimesinin j dokümanındaki önemi büyüktür, düşükse küçüktür.

Ters doküman frekansı (IDF): N doküman kümesindeki toplam doküman sayısı, df_i ise i kelimesinin geçtiği doküman sayısıdır (doküman frekansı, document frequency). IDF, N'nin df_i'ye oranının logaritmasıdır:

$$idf_i = \log \frac{N}{df_i}$$

- i kelimesi kümedeki çok sayıda dokümanda geçiyorsa df_i büyük, IDF küçük olur.
- i kelimesi az sayıda dokümanda geçiyorsa df_i küçük, IDF büyük olur.

TF-IDF ağırlığı: TF ve IDF çarpılarak i kelimesi için bir ağırlık bulunur:

$$w_{ij} = tf_{ij} \times idf_i$$

Boolean modelde dokümanın iç temsili, kelime varsa 1, yoksa 0 olan bir vektördü. Vektör uzayı modelinde ise bu vektörde birinci kelimeden $|V|$'inci kelimeye kadar her kelime için TF-IDF değeri konur. Yani doküman vektöründeki ağırlıklar TF-IDF ile belirlenir.

11.4 Sorgunun Temsili ve Kosinüs Benzerliği

Dokümanlar nasıl tanımlandıysa sorgular da aynı şekilde, toplam ayırt edici kelime sayısı boyutunda ($|V|$) bir vektör olarak tanımlanır. Sonra aynı uzunluktaki bu iki vektör arasındaki uzaklığa bakılır. Bunun için kosinüs benzerliği kullanılır. Daha önce kosinüs benzerliğinin iki doküman arasındaki uzaklık için kullanıldığı söylenmişti; burada vektörlerden biri sorgu, diğeri dokümandır.

Kosinüs benzerliği hesaplanırken payda iki vektörün iç çarpımı (dot product) alınır. Paydada her vektörün uzunluğu bulunur ve bu uzunluklar çarpılır:

$$cosine(d_j, q) = \frac{\langle d_j \cdot q \rangle}{\|d_j\| \times \|q\|} = \frac{\sum_{i=1}^{|V|} w_{ij} \times w_{iq}}{\sqrt{\sum_{i=1}^{|V|} w_{ij}^2} \times \sqrt{\sum_{i=1}^{|V|} w_{iq}^2}}$$

> **Hocanın vurgusu:** Kosinüs benzerliği formülüyle çok karşılaşacaksınız; bilmenizde ve hatırlamanızda fayda var.

11.5 Örnek: hardware ve software Sorgusu

Sözlükte üç kelime vardır: hardware, software, users. Dokuz doküman vardır ve vektörleri slaytta verilmişti:

Doküman	hardware	software	users
A1	1	0	0
A2	0	1	0
A3	0	0	1
A4	1	1	0
A5	1	0	1
A6	0	1	1
A7	1	1	1
A8	1	0	1
A9	0	1	1

Sorgu hardware ve software'dir. Hoca iki soru sordu: Boolean model kullanılsa hangi dokümanlar gelir, vektör uzayı modeli kullanılsa hangi sonuçlar gelir?

Boolean model: Sorgu vektöründe birinci kelime (hardware) için 1, ikinci kelime (software) için 1 vardır. Üçüncü kelime hakkında bilgi yoktur.

- "hardware AND software": Boolean model tam eşleşmeyi getirir, yani sorguyu doğru yapan dokümanları. Bunlar yalnız A4 ve A7'dir. Diğerleri sorguyu doğru yapmaz.
- "hardware OR software": İkisinin de 0 olduğu A3 dışındaki bütün dokümanlar gelir.

Vektör uzayı modeli: Kosinüs benzerliği bir vektörü bir vektörle karşılaştırdığı için sorgu mutlaka vektör olarak tanımlanmalıdır ve her konuma 0 ya da 1 yazılmalıdır. Soruda hardware ve software dışında bilgi verilmediği için üçüncü kelimeye dolaylı olarak 0 konur: $q = (1, 1, 0)$. Sorgu dokuz dokümanla karşılaştırılır. Kosinüs benzerliğinin verdiği değerlere göre dokümanlar sorguya benzerlik sırasıyla verilebilir.

Değerler slaytta gösterildi, hoca sözlü okumadı. Hesap (ara adımlar eklendi, python ile doğrulandı):

- $\|q\| = \sqrt{2}$.
- A4: $q \cdot A4 = 2$, $\|A4\| = \sqrt{2}$, kosinüs $= \frac{2}{\sqrt{2}\sqrt{2}} = 1$.
- A7: $q \cdot A7 = 2$, $\|A7\| = \sqrt{3}$, kosinüs $= \frac{2}{\sqrt{2}\sqrt{3}} \approx 0{,}816$.
- A1 ve A2: iç çarpım 1, uzunluk 1, kosinüs $= \frac{1}{\sqrt{2}} \approx 0{,}707$.
- A5, A6, A8, A9: iç çarpım 1, uzunluk $\sqrt{2}$, kosinüs $= \frac{1}{2} = 0{,}5$.
- A3: iç çarpım 0, kosinüs 0.

Sıralama: A4 > A7 > A1 = A2 > A5 = A6 = A8 = A9 > A3. Boolean AND yalnız iki doküman döndürürken vektör uzayı modeli bütün dokümanları sorguya benzerliklerine göre sıralar.

12. Diğer Benzerlik Fonksiyonları ve İlgililik Geri Bildirimi

12.1 Okapi Benzerliği

Kosinüs benzerliği tek benzerlik fonksiyonu değildir. Okapi benzerliği (Okapi similarity) adlı ölçü de yüksek kaliteli sonuç verir. Kümeleme bölümünde farklı uzaklık ölçüleri görüldüğü gibi, benzerlik için de kosinüs ve Okapi gibi farklı ölçüler vardır.

> **Hocanın vurgusu:** Okapi'yi ezberlemenize gerek yok, yalnızca farklı benzerlik fonksiyonlarının olduğunu bilin. Kosinüs benzerliği ise önemli.
>
> **Ek bilgi (derste anlatılmadı):** Okapi benzerliği, BM25 adıyla bilinen sıralama fonksiyonunun temelidir. Kosinüsten farklı olarak terim frekansının etkisini doyuran (sınırlayan) bir terim ve doküman uzunluğuna göre normalleştirme içerir. Bu yüzden uzun dokümanların yalnızca uzun oldukları için avantaj kazanması azalır.

12.2 İlgililik Geri Bildirimi (Relevance Feedback)

Hoca ilgililik geri bildirimini önce İngilizce, sonra Türkçe anlattı:

1. Kullanıcı ilk başta bir anahtar kelime girer. IR motoru bunu dokümanlara uygular ve bir sonuç listesi döndürür.
2. Motor, gösterdiği listeden ilgili (relevant) bulduğu sonuçların yanına bir işaret koymasını ister. Kullanıcı ilgili olduğunu düşündüğü bağlantıları işaretler.
3. Arama tekrarlanırken motor, işaretlenen bağlantıların gösterdiği dokümanlara gider ve bu dokümanlarda çokça tekrar eden, onları temsil edebilecek kelimeleri çıkarır.
4. Bu kelimeleri kullanıcının ilk girdiği anahtar kelimenin yanına ekler ve aramayı yeniden yapar.

Bu şekilde yapılan aramalarda kullanıcının aramasına daha kaliteli sonuçlar bulunabildiği ortaya konmuştur. İlgililik geri bildirimi kullanan IR motorları buna dayanır.

Örnek Sorular

Soru 1 (derste sözlü, 00:40:08–00:44:15): Yedi ikili niteliği olan iki veri noktası veriliyor: $x_1 = (1,1,1,0,1,0,0)$, $x_2 = (0,1,1,0,0,1,0)$. Aralarındaki uzaklığı (a) basit eşleşme katsayısıyla, (b) 1 durumunun daha önemli olduğu kabul edilerek Jaccard katsayısıyla hesaplayın.

> **Çözüm:** Karışıklık matrisi: $a = 2$, $b = 2$, $c = 1$, $d = 2$. (a) $\frac{b+c}{a+b+c+d} = \frac{3}{7} \approx 0{,}429$. (b) $\frac{b+c}{a+b+c} = \frac{3}{5} = 0{,}6$. Hoca (b)'de $\frac{3}{6}$ dedi; doğru payda $a+b+c = 5$'tir (python ile doğrulandı).

Soru 2 (derste sözlü, 00:54:38–00:57:44): 900 dokümanlı örnekte birinci küme (250 science, 20 sports, 10 politics) ile üçüncü küme (30 science, 100 sports, 210 politics) karşılaştırıldığında hangisinin entropisi ve saflığı daha yüksektir? Neden?

> **Çözüm:** Birinci kümede öğelerin büyük çoğunluğu tek etikettedir. Belirsizlik düşük olduğu için entropi düşük (0,589), dağılım homojen olduğu için saflık yüksektir (0,893). Üçüncü kümede öğeler sports ve politics arasında dağılmıştır. Entropi daha yüksek (1,257), saflık daha düşüktür (0,617).

Soru 3 (derste sözlü, 02:00:52–02:04:09): Sözlük {hardware, software, users} ve Bölüm 11.5'teki dokuz doküman veriliyor. Sorgu hardware ve software'dir. Boolean model kullanılırsa hangi dokümanlar gelir? Vektör uzayı modeli (kosinüs benzerliği) kullanılırsa sonuçlar nasıl sıralanır?

> **Çözüm:** Boolean modelde "hardware AND software" yalnız A4 ve A7'yi, "hardware OR software" A3 dışındaki bütün dokümanları getirir. Vektör uzayı modelinde sorgu $q = (1,1,0)$ alınır. Kosinüs değerleri: A4 = 1; A7 ≈ 0,816; A1 = A2 ≈ 0,707; A5 = A6 = A8 = A9 = 0,5; A3 = 0. Hoca sonuçları kosinüse göre sıralı vermeyi söyledi; sayısal değerler python ile hesaplandı.

Dersin Toparlaması

Dersin ilk yarısında k-means'in iki eksiği, yani küme sayısının baştan verilmesi ve eliptik olmayan şekilleri bulamaması, hiyerarşik kümelemeyle aşıldı. Agglomerative kümeleme her noktayı tek elemanlı küme sayıp en yakın kümeleri birleştirerek dendrogramı kurar. Dendrogramın aşamaları kalite ölçüleriyle karşılaştırılıp en iyisi seçilir. Çok elemanlı kümeler arasındaki uzaklık için tek, tam, ortalama bağlantı ve merkez stratejileri, noktalar arasındaki uzaklık için de veri türüne göre Minkowski ailesi, Chebyshev, basit eşleşme, Jaccard ve kosinüs görüldü. Strateji ve fonksiyon parametrik seçilip denenmelidir. Kümeleme kalitesi etiket yoksa küme içi ve kümeler arası uzaklıkla, etiket varsa entropi ve saflık gibi ölçülerle değerlendirilir.

İkinci yarıda web içerik madenciliğinin ilk bölümü olan bilgiye erişime geçildi. IR'nin yapısız veri ve sorguyla çalışması veritabanından ayrıldı; web aramasının hiperlink, ölçek, yanıt süresi ve spam sorunları sayıldı. IR modelinin dört bileşeni, mimarisi ve sorgu türlerinden sonra Boolean model (tam eşleşme) ile TF-IDF ağırlıklı vektör uzayı modeli (kosinüs benzerliğiyle sıralama) karşılaştırıldı. Hoca bir sonraki aşamada metin ön işleme (text pre-processing) konusunun ayrıntılarına geçileceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.